1. **To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.**

   Task -1: Create Project Structure as follows:

```
OSSP/
│
├── .git/                # Git repository (created automatically)
├── .gitignore           # Files/folders Git should ignore
├── README.md            # Project description and instructions
├── LICENSE              # Project license (optional)
├── Makefile             # Build instructions
│
├── src/                 # Source code
│   ├── main.c
│   ├── shell.c
│   ├── parser.c
│   ├── executor.c
│   └── builtin.c
│
├── include/             # Header files
│   ├── shell.h
│   ├── parser.h
│   ├── executor.h
│   └── builtin.h
│
├── obj/                 # Object files (.o) (generated)
│
├── bin/                 # Executable file (generated)
│   └── my_shell
│
├── docs/                # Documentation
│   ├── design.md
│   └── report.pdf
│
├── tests/               # Test programs
│   ├── test_parser.c
│   └── test_executor.c
│
├── scripts/             # Helper scripts
│   └── run.sh
│
└── assets/              # Images/screenshots
```

└── architecture.png

Note: For creating this structure use mkdir and touch commands.

```
ashborn@DESKTOP-I4G5Q4N:~/OSSP$ pwd
/home/ashborn/OSSP
ashborn@DESKTOP-I4G5Q4N:~/OSSP$ tree
.
├── LICENSE
├── Makefile
├── README.md
├── assets
│   └── architecture.png
├── bin
│   └── my_shell
├── docs
│   ├── design.md
│   └── report.pdf
├── include
│   ├── builtin.h
│   ├── executor.h
│   ├── parser.h
│   └── shell.h
├── obj
├── scripts
│   └── run.sh
├── src
│   ├── builtin.c
│   ├── executor.c
│   ├── main.c
│   ├── parser.c
│   └── shell.c
└── tests
    ├── test_executor.c
    └── test_parser.c
```

2. **To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing.**

Task-1: Using a manual command understand the fork() and exec() system calls
Task-2: Write a C program to create a new process and accepting the user commands using fork() and exec() system calls.

```
ashborn@DESKTOP-I4G5Q4N:~/OSSP$ ./bin/my_shell
This is the Parent Process.
This is the Child Process.
Parent PID = 65
Child PID = 66
Child PID = 66
ashborn@DESKTOP-I4G5Q4N:~/OSSP$ cat src/main.c
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    pid = fork();   // Create a new process

    if (pid < 0)
    {
        printf("Fork failed!\n");
    }
    else if (pid == 0)
    {
        // Child Process
        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());
    }
    else
    {
        // Parent Process
        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }

    return 0;
}
ashborn@DESKTOP-I4G5Q4N:~/OSSP$
```